

SQL 注入漏洞报告（Lab8：查询 MySQL 和 Microsoft 数据库类型和版本）

1. 漏洞名称

SQL Injection（SQL 注入）— 数据库指纹识别与版本信息泄露（MySQL）

2. 漏洞等级

高危（High）

3. 漏洞描述

该网站商品分类查询功能存在 SQL 注入漏洞。攻击者通过 UNION 联合查询，结合不同数据库的内置系统变量，成功获取了后端数据库的类型和具体版本信息，暴露了系统底层环境。本次实验验证了针对 MySQL 数据库的注入手法。

4. 漏洞位置

请求方式：GET /filter?category=...

漏洞参数：category

5. 漏洞复现过程

第一步：初步测试与 500 报错（发现编码问题）

构造 Payload 尝试注入 MySQL 系统变量 @@version。但在浏览器地址栏直接输入 Payload 时遇到了阻碍。

测试 Payload 1（含特殊字符 #）：

text

Food+%26+Drink'+UNION+SELECT+@@version,+NULL#



图 1：浏览器解析问题导致服务器 500 错误

说明：在 URL 中，# 被称为“锚点（Fragment）”，浏览器会直接截断# 后面的内容，不发送给服务器。导致服务器实际接收到的 SQL 变成了只有 1 列，列数不匹配而直接报错。此步骤证明攻击者在构造 Payload 时需要严格遵守 HTTP 协议中的 URL 编码规则。

第二步：URL 编码绕过与成功回显版本信息

将 Payload 中的注释符 # 进行 URL 编码，替换为 %23。这样浏览器就会将 %23 当作普通字符原样发送给服务器。

最终 Payload（成功）：

text

Food+%26+Drink'+UNION+SELECT+@@version,+NULL%23



图 2：成功获取 MySQL 数据库版本信息

说明：使用了 URL 编码的 %23 后，后端服务器成功解析了完整的 SQL 语句。页面回显了 MySQL 的版本号，证明攻击者成功利用 @@version 系统变量获取了数据库类型（MySQL）及具体的软件版本（8.0.42）。



图 3：详细信息展示

说明：清晰的版本号回显（包含 Ubuntu 操作系统信息），证明攻击者不仅拿到了数据库版本，还同时获取了底层操作系统的环境信息。

第四步：漏洞原理

原 SQL:

sql

```
SELECT * FROM products WHERE category='Food & Drink'
```

拼接后的恶意 SQL:

sql

```
SELECT * FROM products WHERE category='Food & Drink' UNION SELECT @@version,  
NULL#
```

- @@version: 这是 MySQL 和 Microsoft SQL Server 共有的系统变量，用于存储当前数据库软件的版本号。
- # (或 --): SQL 注释符，用于将原本跟在后面的单引号和其他 SQL 逻辑注释掉，保证语法合法。
- 通过该注入，攻击者精确识别出目标使用的数据库类型为 MySQL，版本为 8.0.42，运行在 Ubuntu 20.04 系统上。

6. 漏洞影响

暴露数据库版本信息是信息收集的关键环节:

- 攻击者可针对该特定版本 (MySQL 8.0.42) 搜索已知的高危漏洞 (CVE) 进行攻击。
- 揭示底层操作系统信息 (Ubuntu)，方便攻击者寻找系统层提权漏洞。
- 为后续更深层次的数据库攻击 (如读取文件、写 Webshell) 提供了关键情报。

7. 修复建议

- ① **使用参数化查询 (预编译)**: 彻底阻断 SQL 语句拼接，防止注入。
- ② **屏蔽数据库报错信息**: 后端发生 SQL 语法错误或回显异常时，前端统一返回通用的错误页面 (如 404 或 500)，严禁泄露系统变量或表结构。
- ③ **最小数据库权限**: Web 应用连接数据库的账号必须剥夺读取系统视图 (如 @@version 等全局变量) 的权限。

8. 工具

- Google Chrome / Firefox 浏览器 (直接修改 URL 参数并观察回显)
- PortSwigger Web Security Academy (靶场平台)

